

SGSI-UNCP

Política de Seguridad en el Desarrollo de Software y APIs

Documento Base del Sistema de Gestión
de Seguridad de la Información

Universidad Nacional del Centro del Perú

Índice

- 1 Política de Seguridad en el Desarrollo de Software y APIs
 - 1.1 Objetivo
 - 1.2 Alcance
 - 1.3 Principios de Seguridad en el Desarrollo
 - 1.4 Ciclo de Vida de Desarrollo Seguro (SDLC)
 - 1.5 Gestión de Vulnerabilidades y Parches en Desarrollo
 - 1.6 Seguridad en APIs
 - 1.7 Desarrollo por Terceros
 - 1.8 Responsabilidades
 - 1.9 Excepciones
 - 1.10 Documentos Relacionados

1. Política de Seguridad en el Desarrollo de Software y APIs

Organización: Universidad Nacional del Centro del Perú (UNCP) **Aprobado por:** Oficial de Seguridad y Confianza Digital **Norma:** ISO/IEC 27001:2022 (Controles A.8.25–A.8.30) **Alineamiento:** D-SGSI-08 (Guía de Controles ISO 27002), P-SGSI-04 (Gestión de Cambios), POL-SGSI-05 (Dispositivos Móviles)

1.1. Objetivo

Establecer los principios, requisitos y controles de seguridad que deben integrarse en todas las fases del ciclo de vida de desarrollo de software y APIs en la UNCP, garantizando que las aplicaciones institucionales sean seguras por diseño, por defecto y durante toda su operación.

1.2. Alcance

Esta política aplica a: - Todo desarrollo interno realizado por la OTI. - Todo mantenimiento y evolución del ERP ADESA, Campus Virtual (Moodle), SIGA y sistemas satélites. - Todo desarrollo realizado por terceros (proveedores, consultores) contratados por la UNCP. - Toda API expuesta a consumo interno o externo. - Toda integración entre sistemas (cloud, on-premise, SaaS).

1.3. Principios de Seguridad en el Desarrollo

1.3.1. 3.1 Seguridad por Diseño (Shift-Left)

- Los requisitos de seguridad y privacidad deben definirse en la fase de análisis, no agregarse al final.
- Cada nuevo proyecto o modificación significativa debe incluir un **Análisis de Impacto en la Seguridad** antes de comenzar el desarrollo.
- Las aplicaciones móviles desarrolladas por o para la UNCP deben implementar los controles definidos en la **POL-SGSI-05** para entornos BYOD.

1.3.2. 3.2 Principio de Mínimo Privilegio

- Las aplicaciones deben ejecutarse con los permisos mínimos necesarios (principio de least privilege).
- No se permite que las aplicaciones en producción operen con cuentas de administrador o root.

1.3.3. 3.3 Defensa en Profundidad

- Las aplicaciones deben implementar controles de seguridad en múltiples capas: autenticación, autorización, validación de entrada, cifrado, logging y monitoreo.
-

1.4. Ciclo de Vida de Desarrollo Seguro (SDLC)

1.4.1. 4.1 Fase de Requisitos

- Todo requerimiento funcional debe incluir criterios de aceptación de seguridad.
- Se debe identificar la clasificación de los datos que manejará la aplicación (según POL-SGSI-03).
- Se debe definir el modelo de amenazas (threat modeling) mediante metodología STRIDE o similar.

1.4.2. 4.2 Fase de Diseño

- Arquitectura de seguridad documentada (diagrama de flujo de datos, puntos de autenticación, almacenamiento de datos).
- Revisión de arquitectura por el Oficial de Seguridad antes de iniciar la codificación.
- Definición de controles criptográficos (cifrado en reposo y en tránsito, gestión de claves).

1.4.3. 4.3 Fase de Codificación

4.3.1 Estándares de Codificación Segura Todo el código debe seguir buenas prácticas de codificación segura: - Validación de entrada (sanitización, parámetros preparados para SQL). - Codificación de salida (prevención de XSS). - Manejo seguro de sesiones (tokens, expiración). - Control de acceso en cada endpoint (no confiar solo en el frontend). - Manejo seguro de errores (sin exposición de stack traces al usuario).

4.3.2 Escaneo de Seguridad

- **SAST (Static Application Security Testing):** Integrado en el pipeline CI/CD. Todo commit debe ser escaneado antes de fusionarse a la rama principal.
- **Umbral de aceptación:**
 - **Crítico (CVSS 9.0-10.0):** Bloquea el despliegue. Debe corregirse antes de pasar a producción.
 - **Alto (CVSS 7.0-8.9):** Debe corregirse antes del despliegue a producción o tener una justificación aceptada por el Oficial de Seguridad.
 - **Medio (CVSS 4.0-6.9):** Debe registrarse en el backlog técnico y corregirse dentro de los 30 días siguientes.
 - **Bajo (CVSS 0.1-3.9):** Debe registrarse en el backlog técnico.

4.3.3 Análisis de Dependencias

- Se debe mantener un **SBOM (Software Bill of Materials)** actualizado de cada aplicación, listando todas las librerías y dependencias con sus versiones.
- El pipeline CI/CD debe rechazar cualquier dependencia con vulnerabilidades conocidas de severidad Crítica o Alta.
- Las licencias de las dependencias deben ser compatibles con los términos de uso de la UNCP.

1.4.4. 4.4 Fase de Pruebas

Tipo de Prueba	Frecuencia	Herramientas Sugeridas	Responsable
SAST (estático)	Por commit / diario (CI/CD)	SonarQube, Semgrep, Fortify	OTI (Desarrollo)
SCA (dependencias)	Por commit / diario (CI/CD)	OWASP Dependency Check, Snyk	OTI (Desarrollo)
DAST (dinámico)	Trimestral	OWASP ZAP, Burp Suite	OTI (Seguridad)
Pruebas de penetración	Anual (aplicaciones críticas)	Consultora externa	Oficial de Seguridad
Pruebas de integración de seguridad	Por release	Automatizadas en CI/CD	OTI (Desarrollo)

1.4.5. 4.5 Fase de Despliegue

- El despliegue a producción debe realizarse siguiendo el procedimiento de **Gestión de Cambios (P-SGSI-04)**.
- Los entornos de producción, desarrollo y pruebas deben estar **separados físicamente o lógicamente** (control A.8.31).
- Las credenciales y secretos (API keys, contraseñas de BD) no deben estar hardcodeados en el código. Deben gestionarse mediante un **cofre de secretos** (vault).
- Los datos de producción **nunca** deben utilizarse en entornos de desarrollo o pruebas. Se deben usar datos sintéticos o enmascarados.

1.4.6. 4.6 Fase Operativa

- Monitoreo continuo de la aplicación en producción (logs de errores, intentos de acceso no autorizados, rendimiento).
- Las vulnerabilidades detectadas en producción deben gestionarse según la sección 5.
- Actualización periódica de dependencias (al menos trimestral).

1.5. Gestión de Vulnerabilidades y Parches en Desarrollo

Tipo	Plazo de Corrección	Responsable
Vulnerabilidad Crítica (CVSS ≥ 9.0)	48 horas	OTI (Desarrollo) + OTI (Seguridad)
Vulnerabilidad Alta (CVSS 7.0-8.9)	15 días calendario	OTI (Desarrollo)
Vulnerabilidad Media (CVSS 4.0-6.9)	45 días calendario	OTI (Desarrollo)
Vulnerabilidad Baja (CVSS < 4.0)	Próximo release planificado	OTI (Desarrollo)

1.6. Seguridad en APIs

1.6.1. 6.1 Requisitos Obligatorios

- Toda API debe exponerse a través del **API Gateway** designado (TRV-02) para garantizar control de acceso, rate-limiting, logging y auditoría centralizados.
- **Autenticación:** Uso obligatorio de OAuth 2.0 / OIDC para APIs que exponen datos institucionales.
- **Transporte:** HTTPS exclusivamente con TLS 1.2 o superior. Prohibido HTTP plano.
- **Rate Limiting:** Límite de solicitudes por cliente para prevenir abusos y ataques de denegación de servicio.
- **Validación:** Validación de esquema (JSON Schema / OpenAPI) en el API Gateway.
- **Logging:** Toda solicitud y respuesta debe ser registrada para auditoría.

1.6.2. 6.2 APIs Públicas vs. Internas

Tipo	Autenticación	Rate Limiting	Documentación
Interna (solo redes UNCP)	OAuth 2.0 o API Keys	1000 req/min	OpenAPI interna
Externa (accesible desde Internet)	OAuth 2.0 + MFA (si aplica)	100 req/min	OpenAPI pública

1.7. Desarrollo por Terceros

Cuando un proveedor externo desarrolle software para la UNCP: - El contrato debe incluir los requisitos de esta política como anexo obligatorio. - El código fuente y la documentación son propiedad de la UNCP y deben entregarse al finalizar el contrato. - El proveedor debe entregar el SBOM actualizado de todas las dependencias utilizadas. - La UNCP se reserva el derecho de auditar el proceso de desarrollo del proveedor. - El proveedor debe demostrar que realiza escaneos SAST y pruebas de seguridad antes de la entrega.

1.8. Responsabilidades

Rol	Responsabilidad
Desarrollador	Escribir código seguro siguiendo los estándares definidos; corregir vulnerabilidades detectadas en sus aplicaciones.
Líder Técnico / Arquitecto	Revisar la arquitectura de seguridad; asegurar el cumplimiento de esta política en el equipo.
OTI (Seguridad)	Operar las herramientas SAST/DAST/SCA; revisar reportes de vulnerabilidades; aprobar excepciones.
Oficial de Seguridad	Definir y mantener esta política; aprobar el modelo de amenazas de aplicaciones críticas.
Proveedor Externo	Cumplir los requisitos de esta política; entregar código seguro + SBOM.

1.9. Excepciones

Cualquier desviación de esta política debe ser documentada y aprobada por el Oficial de Seguridad. Las excepciones temporales deben tener una fecha de vencimiento y un plan de remediación.

1.10. Documentos Relacionados

Código	Nombre
D-SGSI-08	Guía de Implementación de Controles (ISO 27002)
P-SGSI-04	Procedimiento de Gestión de Cambios en TI
P-SGSI-06	Procedimiento de Gestión de Seguridad con Proveedores
POL-SGSI-05	Política de Seguridad para Dispositivos Móviles del Personal
POL-SGSI-03	Política de Clasificación de la Información y Respaldos

__**